# ADR-001：采用模块化单体架构 (Modular Monolith)

## 状态

已接受

## 背景

项目工期极其受限（仅10周），团队由4名对复杂架构经验有限的学生组成。系统需要涵盖 User、Event、Resource 和 Notice 四大核心业务模块，首要目标是保证极高的交付确定性与后期的可维护性。

## 决策

我们决定放弃微服务方案，采用模块化单体架构。系统在物理上作为一个整体进程部署，但在逻辑代码层面按照业务领域进行严格的分包隔离（Package-by-Feature）。

## 理由

1. **交付确定性**：避开了微服务带来的分布式事务、服务发现、网络通信等复杂“基建”问题，确保10周内能拿出可运行的 MVP 产品。
2. **团队匹配度**：契合学生团队现有的 Java 技术栈基础，本地 IDEA 直接全量联调，降低运维成本。
3. **架构演进**：物理单体但逻辑模块化的代码结构，为未来如果业务激增需要“平滑拆分”为微服务留下了清晰的边界。

## 后果

- 正面影响：开发流程极简，模块间调用为纯内存方法调用，性能极高；部署极为简便。
- 负面影响：所有模块共用一个进程，某个模块的严重内存泄漏会导致整个应用崩溃。
- 需要关注的风险：随着代码量增加，必须严格依靠规范防止模块之间产生强耦合的“面条代码”。

## AI 辅助记录

- AI 初稿内容摘要：在不考虑工期的情况下，强烈建议采用 Spring Cloud 微服务架构以保证高扩展性。
- 人工修订内容：明确拒绝微服务，改用模块化单体架构，并强调了“按功能分包”的落地策略。
- 修订理由：AI 的建议过于追求技术前沿，忽略了“10周极短工期”和“4人初学者团队”这两个致命的工程约束。

---

# ADR-002：前端开发语言选择纯 JavaScript (放弃 TypeScript)

## 状态

已接受

## 背景

需要确定前端 Vue 3 项目的开发语言。尽管 TypeScript 能提供强大的类型安全，但当前团队成员普遍缺乏对泛型、复杂类型推导等 TS 高级特性的实战经验。

## 决策

我们决定在前端 Vue 3 框架中，全面采用纯 JavaScript (ES6+) 进行开发，不引入 TypeScript。

## 理由

为了在极其紧张的项目初期抢占开发时间，必须将“降低团队学习成本”的优先级置于“绝对的类型安全”之上，避免大家因为复杂的类型报错而卡住进度。

## 后果

- 正面影响：团队成员上手极快，代码编写更自由、更迅速，初期联调效率大幅提升。
- 负面影响：缺乏编译时的类型检查，对于后端返回的复杂对象，容易出现拼写错误或类型取值错误（如 `undefined` 导致白屏）。
- 需要关注的风险：必须通过完善的 API 文档（Swagger）和严格的 Code Review 机制来人工弥补类型约束的缺失。

## AI 辅助记录

- AI 初稿内容摘要：极力推荐使用 TypeScript，认为这已经是现代前端工程的绝对标配，能大幅降低生产环境 Bug。
- 人工修订内容：改为了纯 JavaScript，并重点加入了由于学习成本导致的权衡考量。
- 修订理由：AI 侧重于长远的技术红利，而人工评审认为在当前短平快的学生项目中，强制引入 TS 的风险远大于收益。

---

# ADR-003：严禁跨模块直接调用 Repository

## 状态

已接受

## 背景

采用了模块化单体架构后，虽然逻辑上分了包（如 `User` 和 `Event` 包），但在同一个工程内，开发者极容易图省事，在一个模块的 Service 里直接注入另一个模块的 Repository 进行查表操作。

## 决策

我们设立工程规范红线：任何模块之间的跨域交互，必须通过对方模块的 `Service` 层公开接口进行。严禁直接调用、操作或关联其他模块的数据库层（Repository/Mapper）。

## 理由

维持模块化单体架构不腐化的核心手段。如果 `Event` 模块直接查 `User` 表，那么一旦 `User` 表结构变更，将导致多个模块级联报错，彻底丧失解耦意义。

## 后果

- 正面影响：保证了各模块内部数据结构（表结构）对外部彻底透明，模块边界极其清晰。
- 负面影响：对于某些极简单的跨域查询（比如通过 userId 查 username），原本一行查表代码现在必须通过 Service 层层转发，略微增加了模板代码量。
- 需要关注的风险：由于全部通过 Service 调用，可能会在不经意间引发 Service 之间的循环依赖（A 依赖 B，B 依赖 A）。

## AI 辅助记录

- AI 初稿内容摘要：仅笼统地提到了遵循 MVC 三层架构，Controller 调 Service，Service 调 Mapper。
- 人工修订内容：硬性追加了“跨业务域严禁跨层调用”的防腐规范。
- 修订理由：AI 遗漏了在单体项目中保持模块独立性的核心痛点，人工修订强化了工程架构的底线约束。

---

# ADR-004：采用 JWT (JSON Web Token) 实现无状态鉴权

## 状态

已接受

## 背景

系统为前后端完全分离架构。由于不依赖传统的模板引擎（如 JSP/Thymeleaf），传统的 Session-Cookie 机制会导致跨域请求处理复杂，且不利于后续扩展。

## 决策

我们决定采用基于 JWT (JSON Web Token) 的无状态身份认证机制。客户端登录成功后保存 Token，并在后续所有请求的 HTTP Header 中携带该 Token 访问受保护接口。

## 理由

JWT 将用户状态保存在客户端，服务端实现彻底的无状态化。这既天然避开了跨站请求伪造 (CSRF) 攻击，又能显著降低服务端的内存开销。

## 后果

- 正面影响：完美契合 RESTful API 设计，前后端联调更清晰；若未来加入移动端小程序，复用度极高。
- 负面影响：Token 一旦签发，在过期前服务端难以主动使其强制失效（除非引入 Redis 做黑名单，这会增加复杂度）。
- 需要关注的风险：Token 载荷（Payload）部分为 Base64 明文编码，绝不能在其中存放密码等敏感信息；需合理设置过期时间。

## AI 辅助记录

- AI 初稿内容摘要：给出了 Session/Cookie、JWT 和 OAuth2.0 三种方案的优缺点对比，建议视情况选择。
- 人工修订内容：直接敲定采用 JWT，排除了 OAuth2.0 的过度设计和 Session 的陈旧机制。
- 修订理由：AI 的回答偏向于综述，人工则直接针对本项目"前后端分离"的特性做出了果断决策。

---

# ADR-005：建立统一的 API 响应格式与全局异常处理

## 状态

已接受

## 背景

在团队并行开发中，如果后端接口返回的数据格式五花八门，或者遇到异常时直接抛出 Tomcat 的 500 默认错误页，前端解析将会彻底崩溃，大量时间会浪费在沟通接口格式上。

## 决策

后端建立统一的 Result 包装类，所有接口固定返回 `{code: 状态码, message: 提示信息, data: 实际数据}` 格式。同时，使用 Spring 的 `@ControllerAdvice` 拦截所有系统级别的抛出异常，并将其转化为该 JSON 格式返回给前端。

## 理由

标准化前后端交互契约。前端可以编写全局的 Axios 响应拦截器统一处理错误逻辑（如 Token 过期弹窗、无权限报错），无需在每个业务页面里重复写判断。

## 后果

- 正面影响：前后端联调体验极其顺滑，代码高度复用。
- 负面影响：后端开发人员需要时刻注意不要将原始数据直接 return，必须包一层 Result。
- 需要关注的风险：全局异常拦截器如果不当心，可能会"吃掉"底层的异常堆栈信息，导致后台控制台看不到报错日志，增加排错难度。

## AI 辅助记录

- AI 初稿内容摘要：提供了一套极其复杂的 HTTP 状态码（200, 201, 400, 401, 403, 500等）配合 RESTful 规范的方案。
- 人工修订内容：放弃了纯粹基于 HTTP 状态码的方案，统一采用 HTTP 200 配合业务自定义 Code（如 20000 成功，50000 失败）的工业界主流做法。
- 修订理由：完全遵循 HTTP 状态码在实际开发中极其繁琐且易产生分歧，人工修订采用了更易于前端处理的业务层状态码封装方案。

---

# ADR-006：使用 Swagger/Knife4j 作为前后端契约基准

## 状态

已接受

## 背景

时间只有 10 周，我们不能等待后端把接口写完、测好之后，前端才开始对接开发，必须实现最大程度的前后端并行推进。

## 决策

我们决定在 Spring Boot 后端工程中集成 Swagger (搭配 Knife4j UI 组件)，并将“契约先行”作为团队铁律：编写任何业务逻辑前，必须先定义好 Controller 及出入参实体，生成可视化 API 文档。

## 理由

有了可视化的接口契约，前端可以据此立刻开始编写 Mock 数据和页面 UI，后端可以独立进行业务逻辑编写。文档与代码实时同步，彻底消灭外部文档（如 Word、Wiki）容易过期滞后的问题。

## 后果

- 正面影响：并行开发效率最大化，文档零维护成本（代码即文档）。
- 负面影响：后端代码中会混杂大量的 `@Api`、`@ApiOperation` 等 Swagger 注解，略显臃肿。
- 需要关注的风险：正式上线前（部署到公网服务器时）必须通过配置关闭 Swagger 页面，否则存在极其严重的数据安全泄露风险。

## AI 辅助记录

- AI 初稿内容摘要：推荐使用 YApi、Postman 或 Apifox 建立独立团队空间来管理 API 文档。
- 人工修订内容：放弃第三方平台，直接采用代码内嵌的 Swagger/Knife4j。
- 修订理由：独立平台需要额外的人工维护精力，很容易出现“代码改了文档没改”的情况，自动化生成的 Swagger 最稳妥。

---

# ADR-007：采用 Docker Compose 统一本地运行与服务器部署环境

## 状态

已接受

## 背景

项目中引入了 MySQL 和 Redis。4 名团队成员使用的操作系统（Windows/Mac）和本地软件版本存在差异。如果各自在本地安装，极易出现“这段代码在我电脑上好好的，拉给你就报错”的玄学问题。

## 决策

我们决定编写 `docker-compose.yml` 文件。将数据库、Redis 等基础设施，以及最终打包后的后端 Spring Boot Jar 文件，全部通过 Docker 容器化编排运行。

## 理由

提供绝对一致的运行时环境。新人拉取代码后，只需执行一行命令 `docker-compose up -d` 即可在本地瞬间拉起完整的项目生态，服务器部署也采用同套配置，实现无缝衔接。

## 后果

- 正面影响：彻底解决环境不一致问题，零配置“开箱即用”。
- 负面影响：全组人员本地电脑必须安装配置 Docker 环境，对低配电脑内存有一点压力。
- 需要关注的风险：必须正确配置 Docker Volume（数据卷）挂载规则，否则容器一旦销毁，MySQL 中的业务数据将彻底丢失。

## AI 辅助记录

- AI 初稿内容摘要：推荐使用 Kubernetes (K8s) 或者 GitLab 自动化构建镜像推送到云端镜像仓库的完整 CI/CD 流程。
- 人工修订内容：砍掉了 K8s 和复杂的镜像仓库推送流，降级为最极简的单机 Docker Compose 编排。
- 修订理由：对于一个单体架构的期末项目，引入 K8s 纯属大炮打蚊子，增加了不必要的运维负担，Docker Compose 足以满足统一环境的需求。